



Magnetospheric Classification

How to teach a computer to read ion spectrograms | ASAP Summer School, KTH
Stockholm, June 2026

Where Is MMS Right Now?

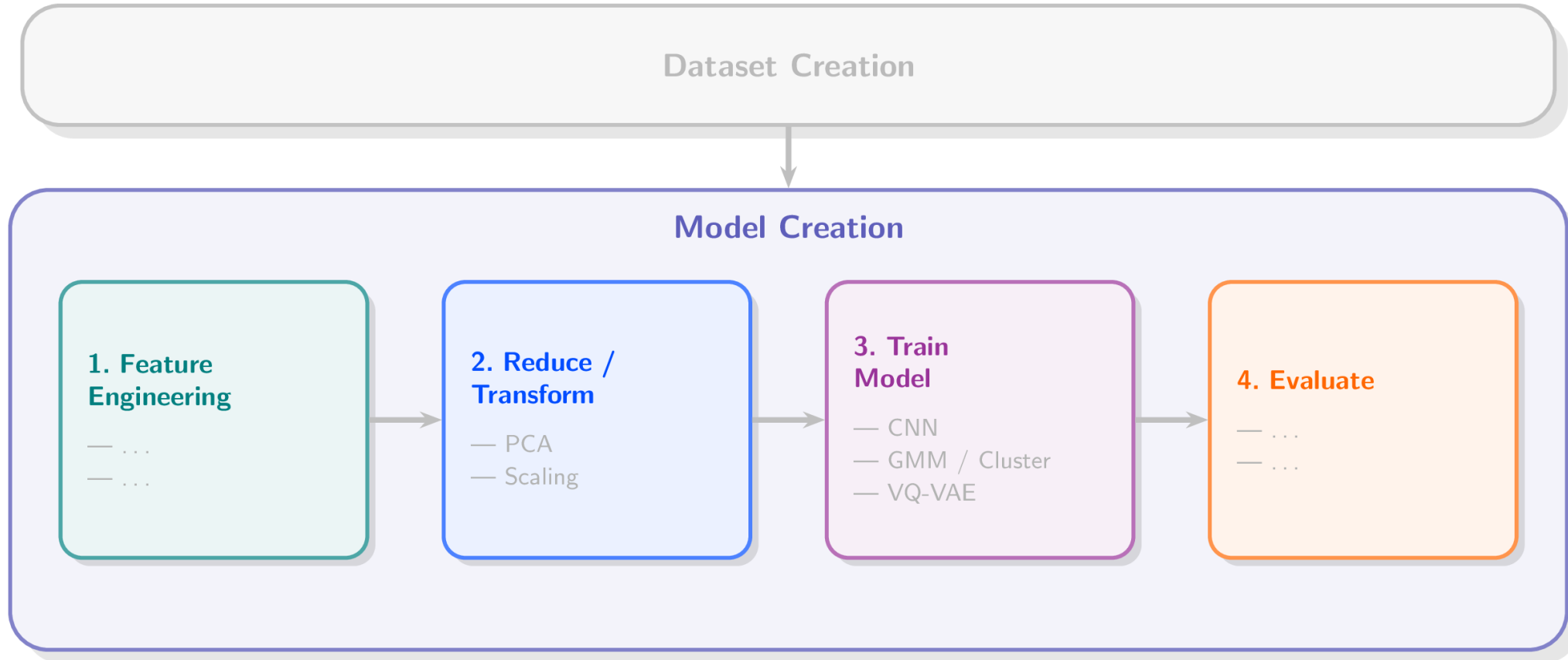
A real-world labelling problem

- MMS orbits Earth every approx. 1 day, crossing 4 plasma regions: Solar Wind, Ion Foreshock, Magnetosheath, Magnetosphere
- SITL scientists label crossings by eye - a trained expert can do this in seconds from a spectrogram
- 8 years of MMS data at ~100 GB/day - human labelling at this scale is impossible

The question that drives everything:

"How would you teach a computer to do what the scientist does by eye?"

The Pipeline





Section 1: Know Your Data(set)

Notebook: 02_data_exploration

The Problem Setup: MMS Ion Spectrograms

What is the data?

- MMS measures ion flux vs. energy (~ 10 eV to 30 keV) every 4.5 seconds
- One measurement = a vector (flux at each energy bin); many measurements = a 2-D spectrogram
- We group N consecutive measurements into a region homogeneous image window.
- Earth Dayside

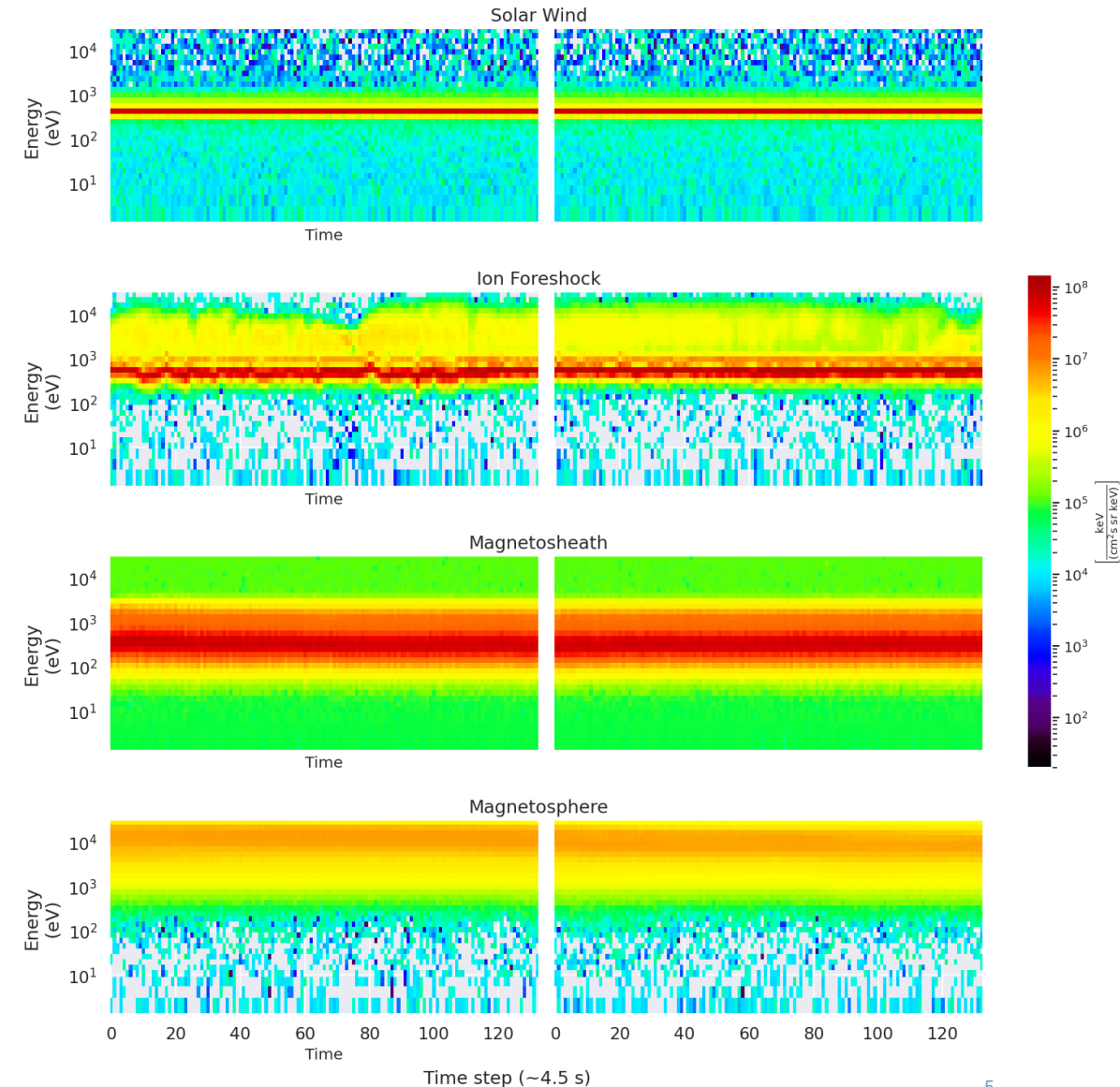
Four plasma regions, four distinct spectral fingerprints:

Solar Wind: narrow, high-energy peak

Ion Foreshock: like SW but with a high-energy tail

Magnetosheath: broad, medium-energy peak

Magnetosphere: flux increases logarithmically with energy



Loading Data and the Windowing Concept

Why use windows, not single measurements?

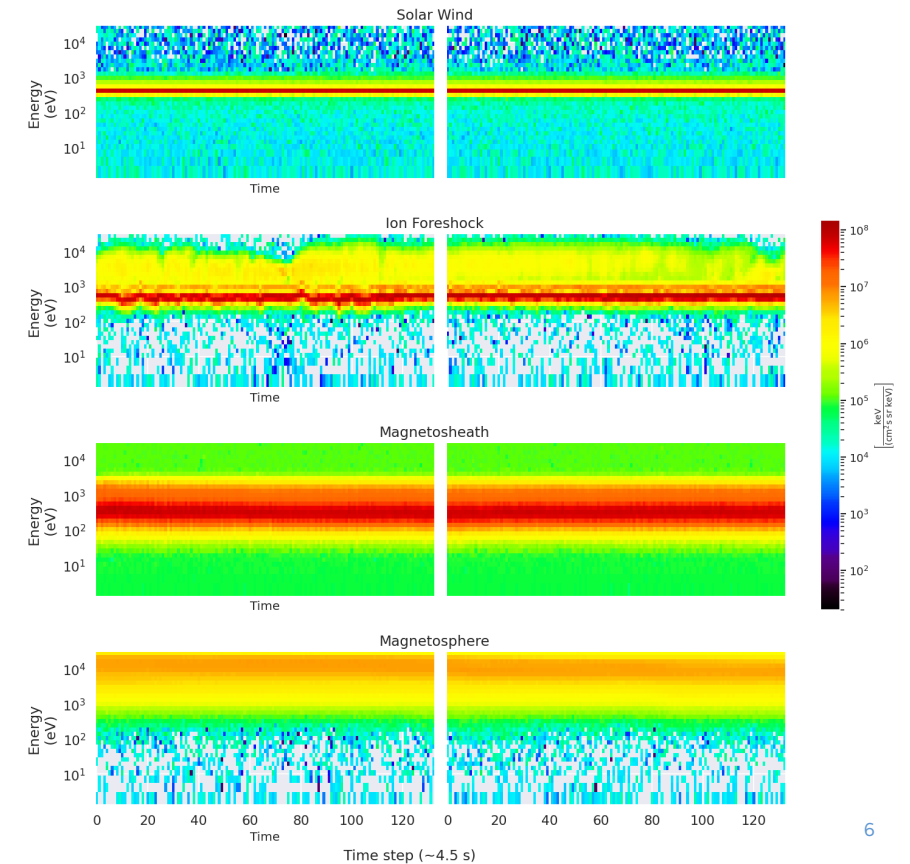
- A single measurement can be noisy – there may be a transient event.
- Grouping consecutive measurement captures the temporal context and averages out the noise
- Looking at time windows is common in human labeling.
- Trade-off: longer windows = more context and less noise, but fewer samples and coarser time resolution.
- Each time-energy pixel is a feature:
 - High-dimensional feature space
 - High degree of regularity -> PCA

SpectrumDataset

- Olshevsky et al. (2021) labeled single measurements
 - We need to group by labels and find areas with consecutive labels.
- Here we will use the **SpectrumDataset** from **SpacePhyML**
 - Windows by consecutive labels



```
from spacephyml.dataset.mms import SpectrumDataset
ds = SpectrumDataset(
    N = 133 # ≈ 10 minutes at 4.5 s cadence
)
```



Class Imbalance: What Happens If We Ignore It?

The problem

- MMS spends most of its time in the Solar Wind
- A classifier trained on this imbalanced data will gain accuracy by predicting the majority class
- Example (right) our dataset,
 - 77% of Samples are either Solar Wind or Magnetosheath
 - A model that only learns these two regions will score high but not be useful (unless we only care about these regions)



The fix

- **Option A:** Downsample the larger classes.
 - Pro: Balanced dataset
 - Con: Discard data
- **Option B:** Upsample the minority classes
 - Pro: More data
 - Con: Data duplication



```
from spacepyml.datasest.mms import SpectrumDataset
ds = SpectrumDataset(
    N = 133,          # ~ 10 minutes at 4.5 s cadence
    samples = 208,    # Nr. Samples per class.
)
```

Preprocessing: Why Log-Scale Matters

The challenge: ion flux spans approximately 8 decades

- Raw values: some energy bins differ by a factor of $1e8$
- $\log_{10}(\text{flux} + 1)$ compresses the dynamic range while preserving the spectral shape

Two strategies compared:

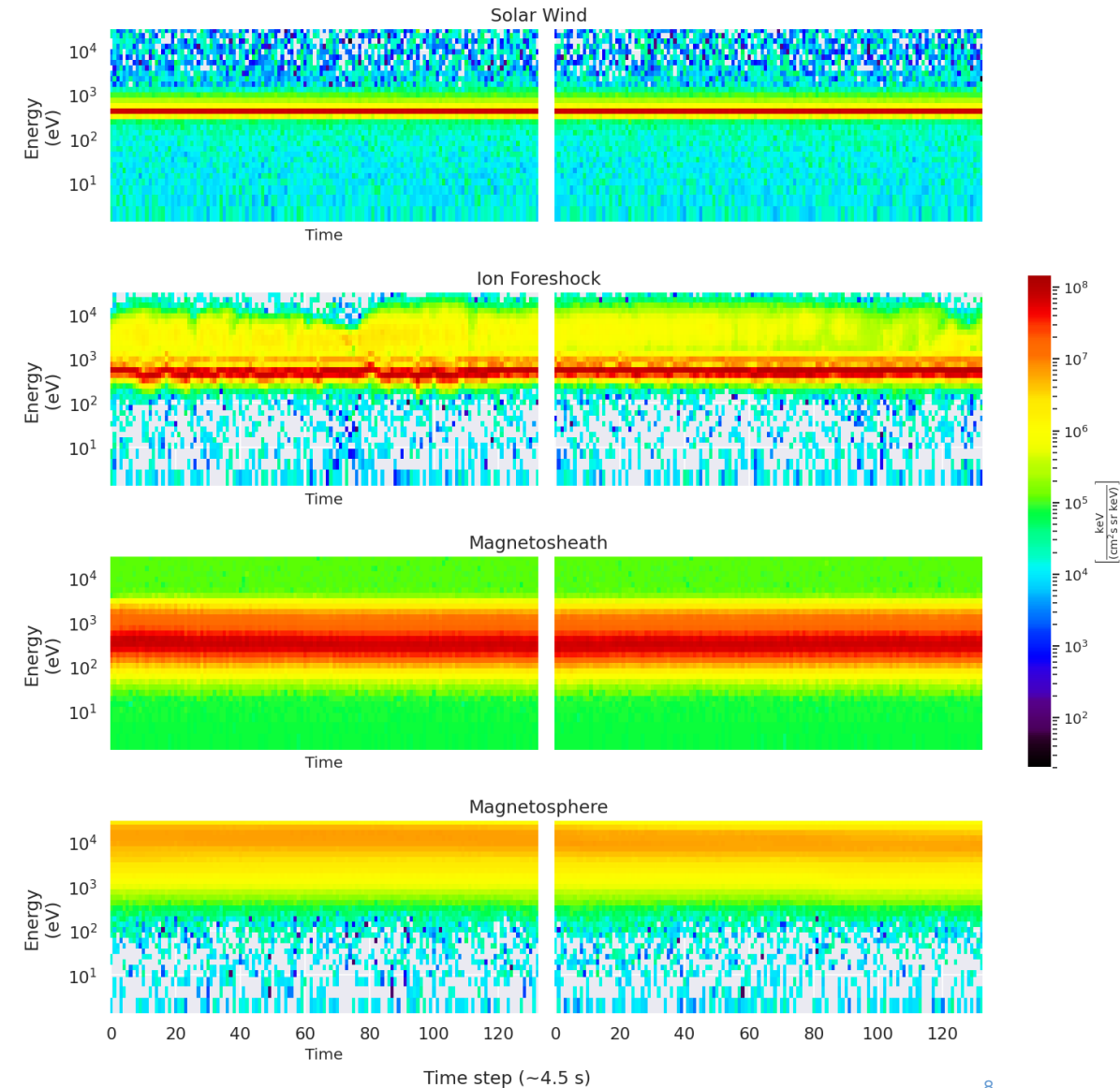
- `log10p1`: applies $\log_{10}(x+1)$ globally
- `minmax`: scales each feature to $[0, 1]$ globally

Critical warning

- Per-feature standardization erases the spectral profile that distinguishes the regions

```
def preprocess(X, scaling):
    if scaling == 'log10p1':
        return np.log10(1. + X)
    elif scaling == 'minmax':
        lo = np.percentile(X, 1)
        hi = np.percentile(X, 99)
        return (np.clip(X, lo, hi) - lo) / (hi - lo)

dsi = SpectrumDataset(N=133, samples=208)
X_flat, _ = dsi[:]
X_scaled = preprocess(X_flat.numpy(), scaling='log10p1')
```

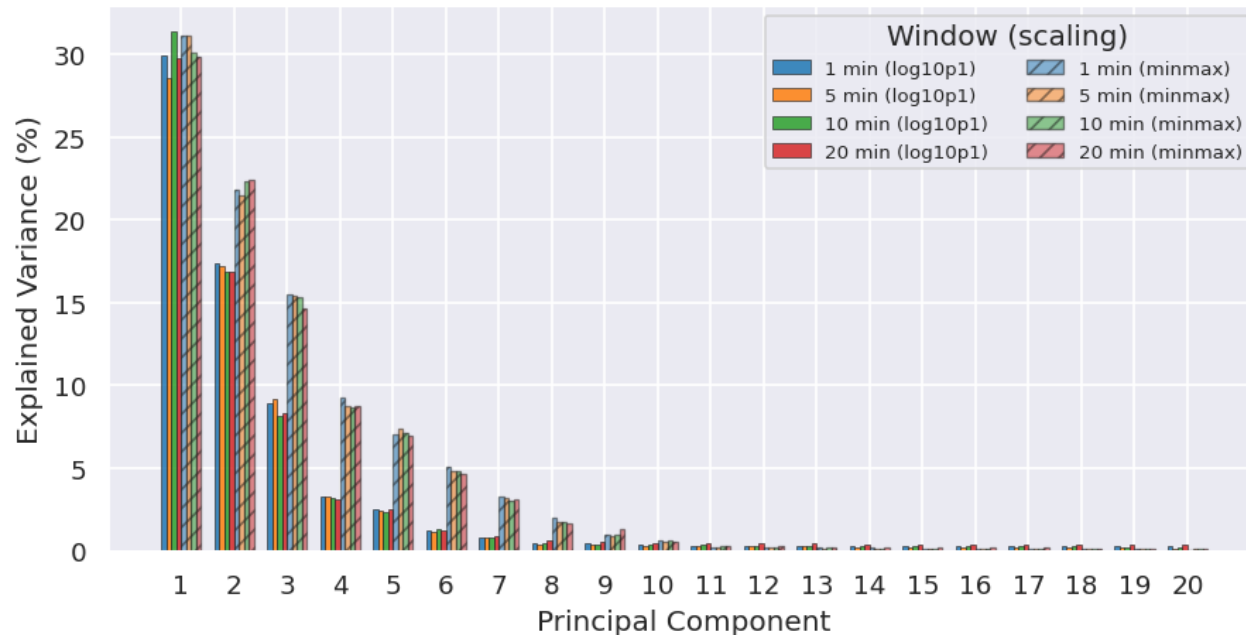


PCA: How Many Dimensions Do We Really Need?

Principal Component Analysis

- PCA finds directions of maximum variance - like finding the axis a cloud of points is most stretched along
- It projects high-dimensional data (32 energy bins x N time steps) down to a few principal components

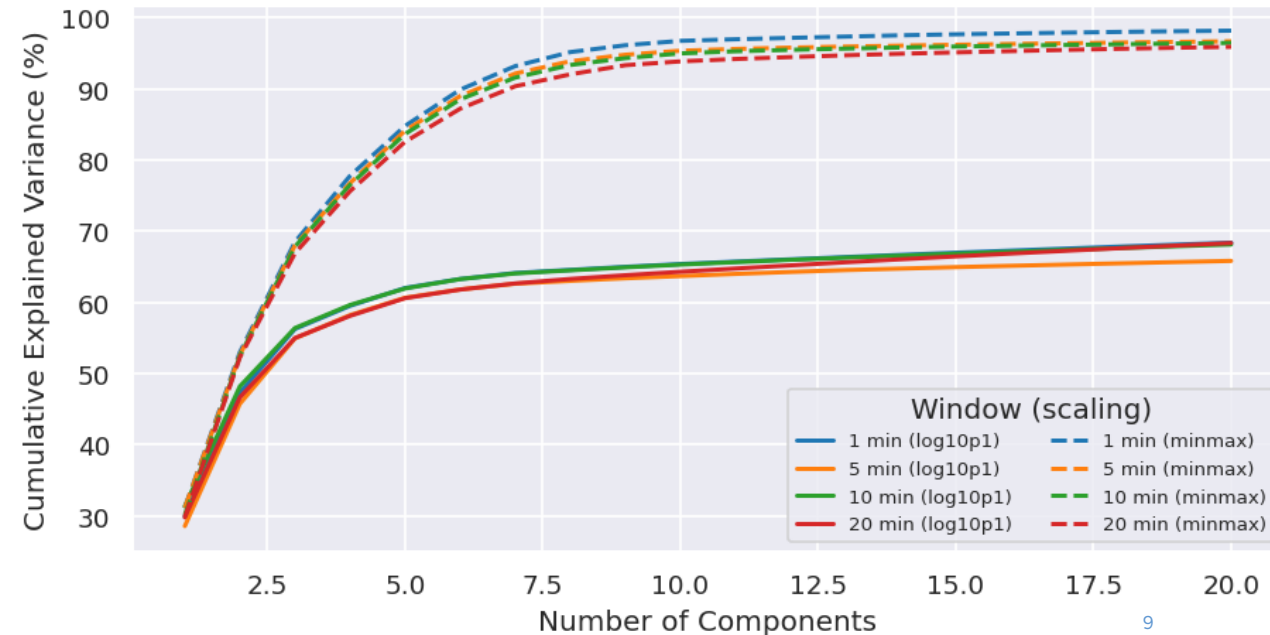
Scree Plot



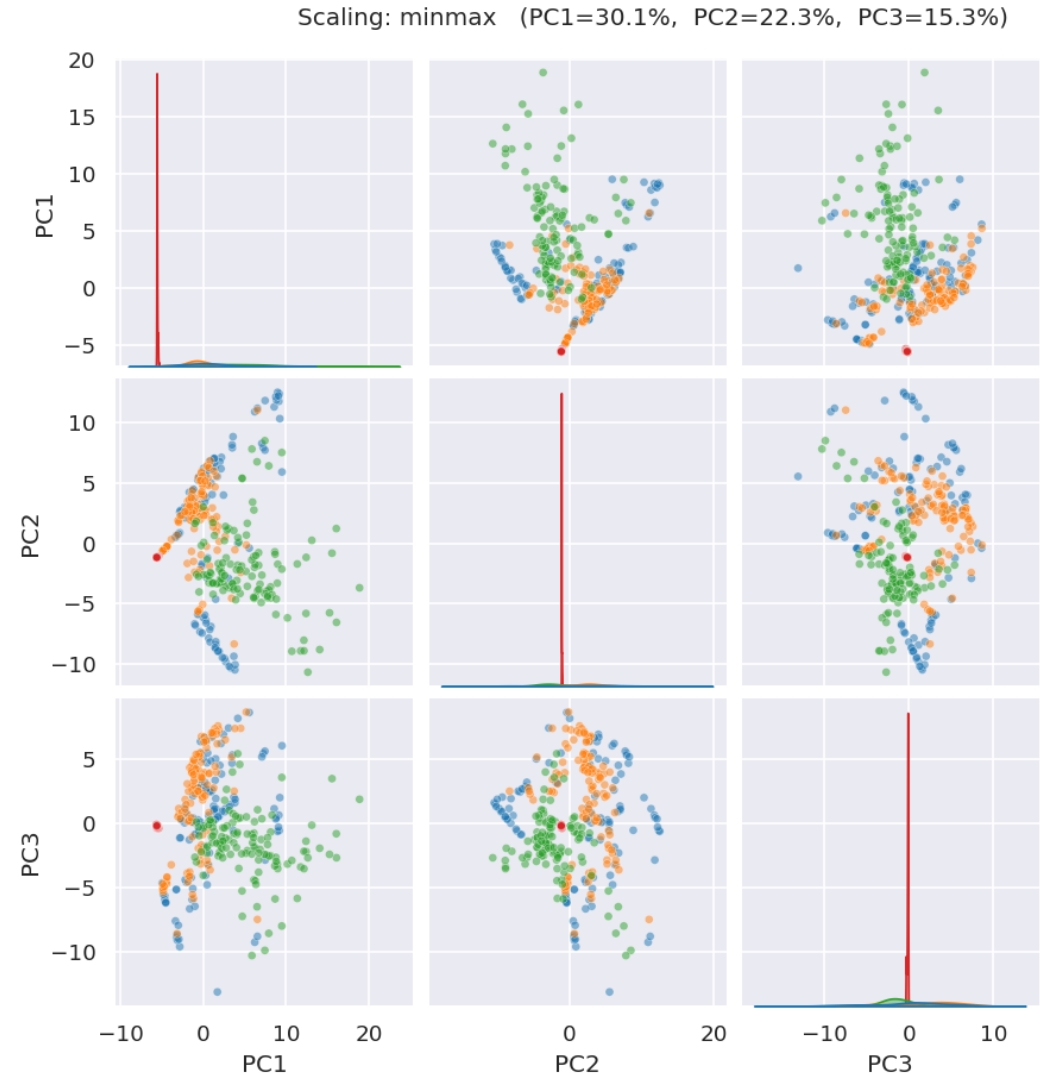
Scree plot across window lengths (1, 5, 10, 20 min)

- Longer windows carry more temporal structure
 - But does not require more components
- Both log10p1 and minmax scaling's agree: most variance is captured in approximately 5-8 components

Cumulative Explained Variance



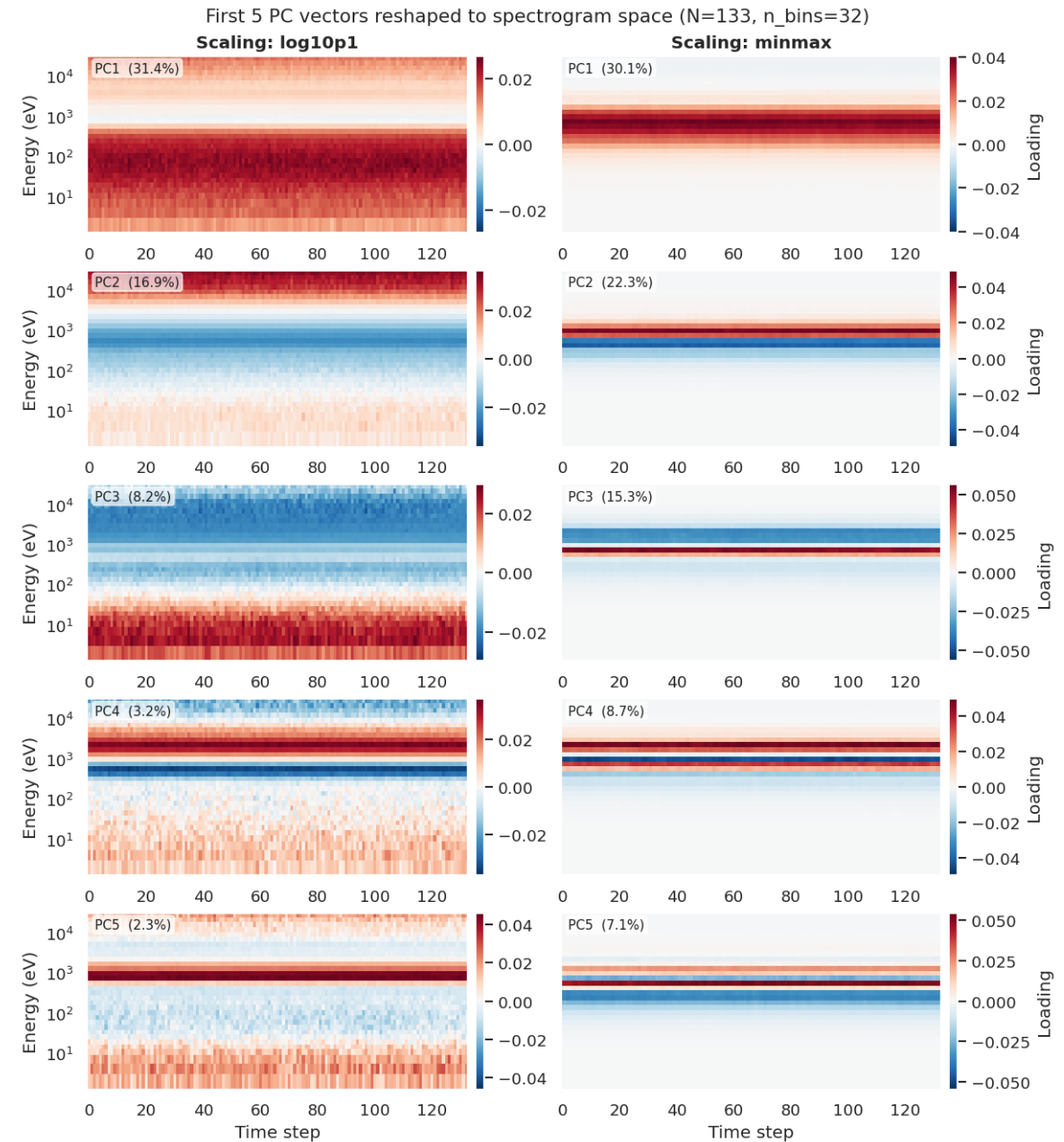
PCA Pair Plot: Regions Already Cluster!



Principal components

The principal components can give insight.

- What is the model looking at?



Summary

1. Load the data (using the SpectrumDataset)
2. Inspect the data (time windows of samples)
 1. Distributions
 2. Mean and Std
3. Inspect class balance/imbalance
4. Consider scale of features
5. High-dimensional but regular?
 1. Consider dimensionality reduction, (PCA)
6. Evaluate feature correlations

Section 2:

Unsupervised

Classification -

Clustering

Notebook: 03_pca_clustering_plasma_regions

What Is Clustering For?

Supervised learning requires labeled data

- In space physics, labeling years of data by hand is expensive and subjective
- Clustering asks: can the data organize itself into meaningful groups without labels?

Two modes of usefulness

- Labeling at scale: Toy-Edens et al. (2024) classified 8 years of MMS data - ~100 GB/day, no human labels
- Discovery: finding structure you did not expect - new plasma populations, transition states

The catch

- Cluster IDs are arbitrary - cluster 0 is not necessarily Solar Wind
- Evaluation requires either label-free metrics (ARI, NMI) or mapping cluster IDs to known labels

Train / Test Split: The Core Discipline

The fundamental question in ML

"How well does the model generalise to data it has never seen?" - not just how well it fits training data

The split

- Training set (~70%): the model, PCA, and scaler see this data and learn from it
- Test set (~30%): held back entirely until evaluation - one honest measure of generalisation
- Analogy: training set = practice problems; test set = the exam
- stratify=y ensures both splits have the same class proportions

Critical concept: data leakage

- Fitting PCA or a scaler on the test set leaks information - the model silently sees test data during training
- Always fit on the training set only, then apply the same transform to both train and test

```
N = 66          # ~ 5 minutes at 4.5 s cadence
SAMPLES = 521   # windows per class (balanced)
RANDOM_STATE = 42 # Set random state for reproducibility
TEST_SIZE = 0.30

ds = SpectrumDataset(
    dataset_path=NC_FILE,
    N=N,
    samples=SAMPLES,
    flatten=True,
)

X_raw, y = ds[:]

X_scaled = np.log10(1 + X_scaled)

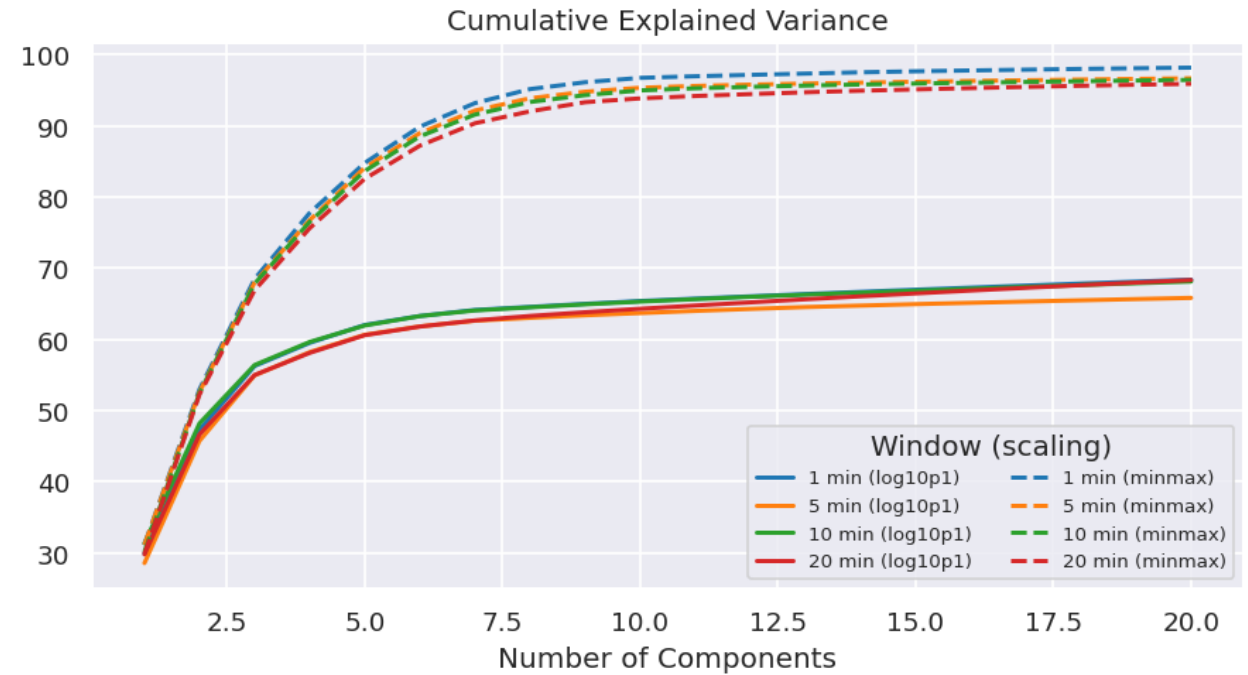
X_train, X_test, y_train, y_test = train_test_split(
    X_raw, y,
    test_size=TEST_SIZE,
    stratify=y,
    random_state=RANDOM_STATE,
)
```

PCA Transform

After feature reduction

- Separation visible in first 2 components
- First 8 components contain > 60% of variance
- Use PCA for feature reduction

Fit only on training data!



```
N_PCS = 8

pca = PCA(n_components=None, random_state=RANDOM_STATE)
pca.fit(X_train)

X_train_pca = pca.transform(X_train)[: , :N_PCS]
X_test_pca  = pca.transform(X_test) [: , :N_PCS]
```


Three Algorithms - Three Assumptions

K-Means

- Minimizes (Euclidean) distance to cluster center
- Hard assignments - each point belongs to exactly one cluster

GMM

- Fits a mixture of Gaussians - elliptical clusters
- Soft (probabilistic) assignments
- Each region gets a probability - useful at transition zones

Agglomerative

- Builds a dendrogram bottom-up - no distributional assumption
- Cannot classify new data without refitting

```
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.mixture import GaussianMixture

# — K-Means —
kmeans = KMeans(n_clusters=N_CLUSTERS,
                n_init=20, max_iter=500,
                random_state=RANDOM_STATE)
kmeans.fit(X_train_pca)
kmeans_test = kmeans.predict(X_test_pca)

# — Gaussian Mixture Model —
gmm = GaussianMixture(n_components=N_CLUSTERS,
                      covariance_type='full',
                      n_init=5, max_iter=300,
                      random_state=RANDOM_STATE)
gmm.fit(X_train_pca)
gmm_test = gmm.predict(X_test_pca)

# — Agglomerative Clustering —
agg = AgglomerativeClustering(n_clusters=N_CLUSTERS,
                              linkage='ward')
all_pca = np.vstack([X_train_pca, X_test_pca])
agg_labels_all = agg.fit_predict(all_pca)
agg_train = agg_labels_all[:len(X_train_pca)]
agg_test = agg_labels_all[len(X_train_pca):]
```

Evaluating Clustering: ARI, AMI, label assignment

The challenge: cluster IDs are arbitrary

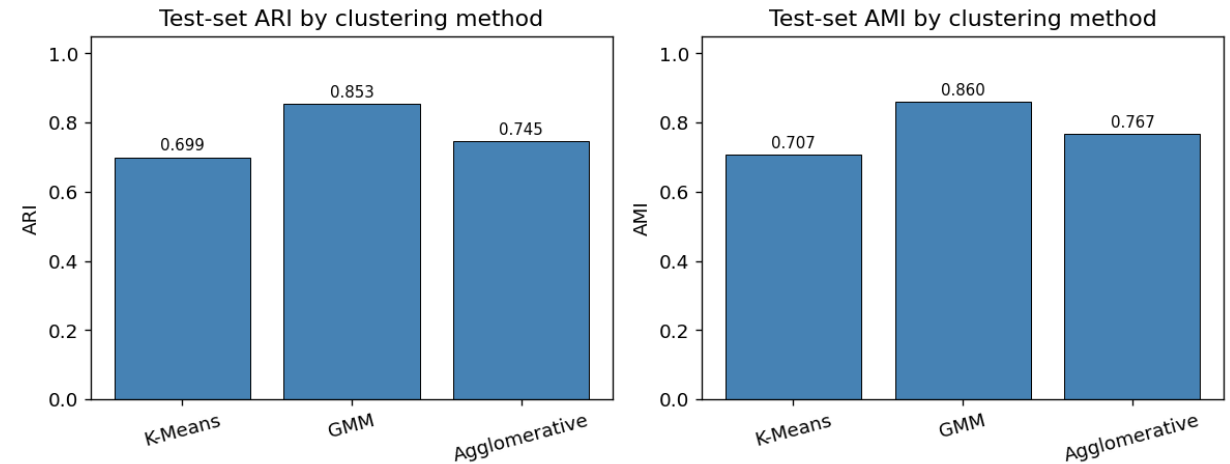
- Cluster 0 may correspond to Solar Wind in one run, Magnetosheath in another
- Accuracy requires a mapping from cluster IDs to region labels first

Label-free metrics (no mapping needed)

- Adjusted Rand Index (ARI):
 - How well do the clusters agree with the labels?
 - Range: ≤ 0 (no better than random) $\rightarrow 1$ (perfect).
- Adjusted Mutual Information (AMI):
 - If we know the cluster, how certain can we be of the label?
 - Range: $-1 \rightarrow 1$.

For confusion matrices from clustering

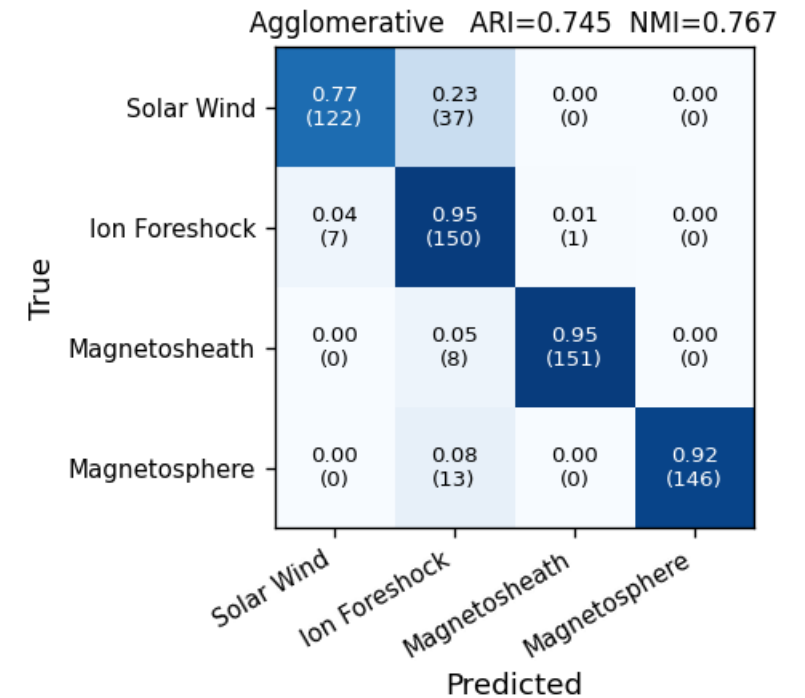
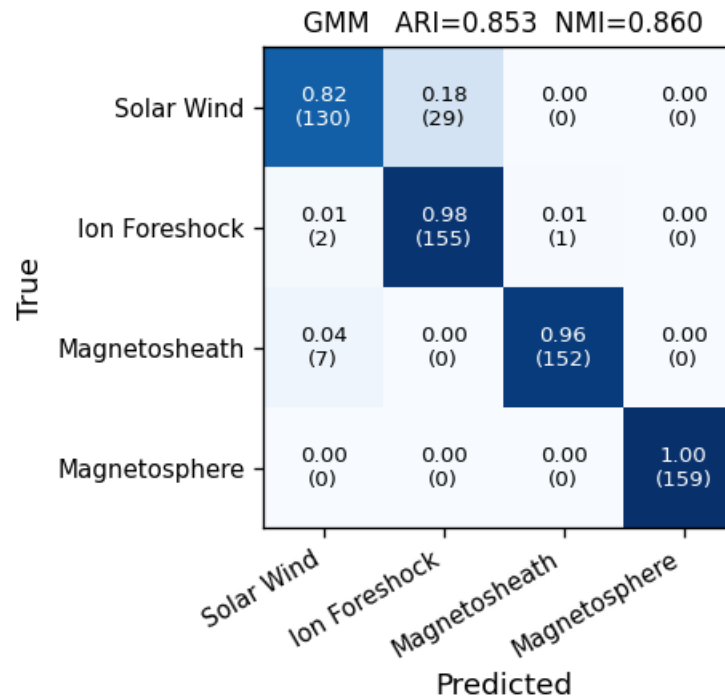
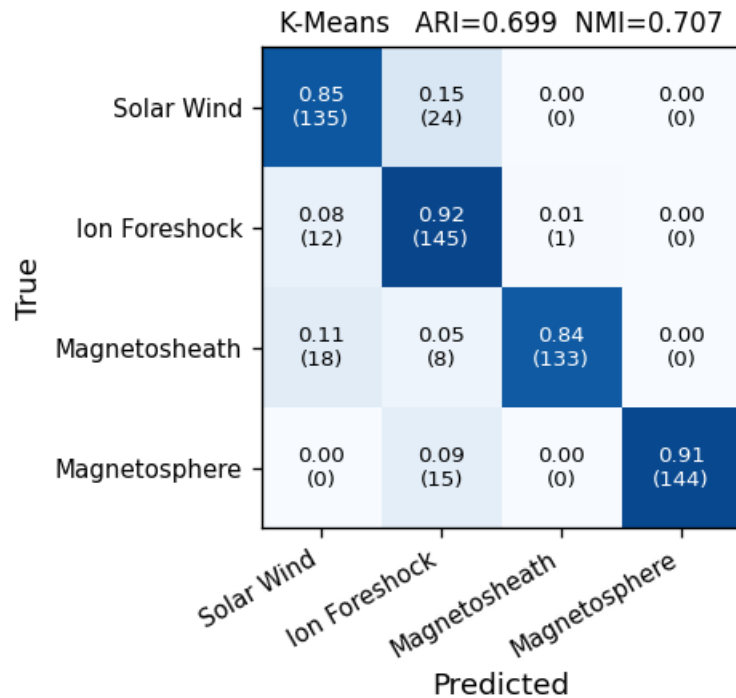
- Use the majority vote to find mapping from cluster IDs to region labels



```
def majority_vote_match(y_true, y_pred):  
    """Remap cluster IDs to best-matching true  
    labels via majority vote."""  
    cm = confusion_matrix(y_true, y_pred)  
    mapping = {pred: true for pred, true in enumerate(cm.argmax(axis=0))}  
    return np.array([mapping.get(p, -1) for p in y_pred])  
  
ari = adjusted_rand_score(y_test, kmeans_test)  
ami = adjusted_mutual_info_score(y_test, kmeans_test)  
kmeans_labels = majority_vote_match(y_test, kmeans_test)
```

The Confusion Matrix: Reading Model Failures

Confusion matrices (row-normalised) — test set, Hungarian label matching

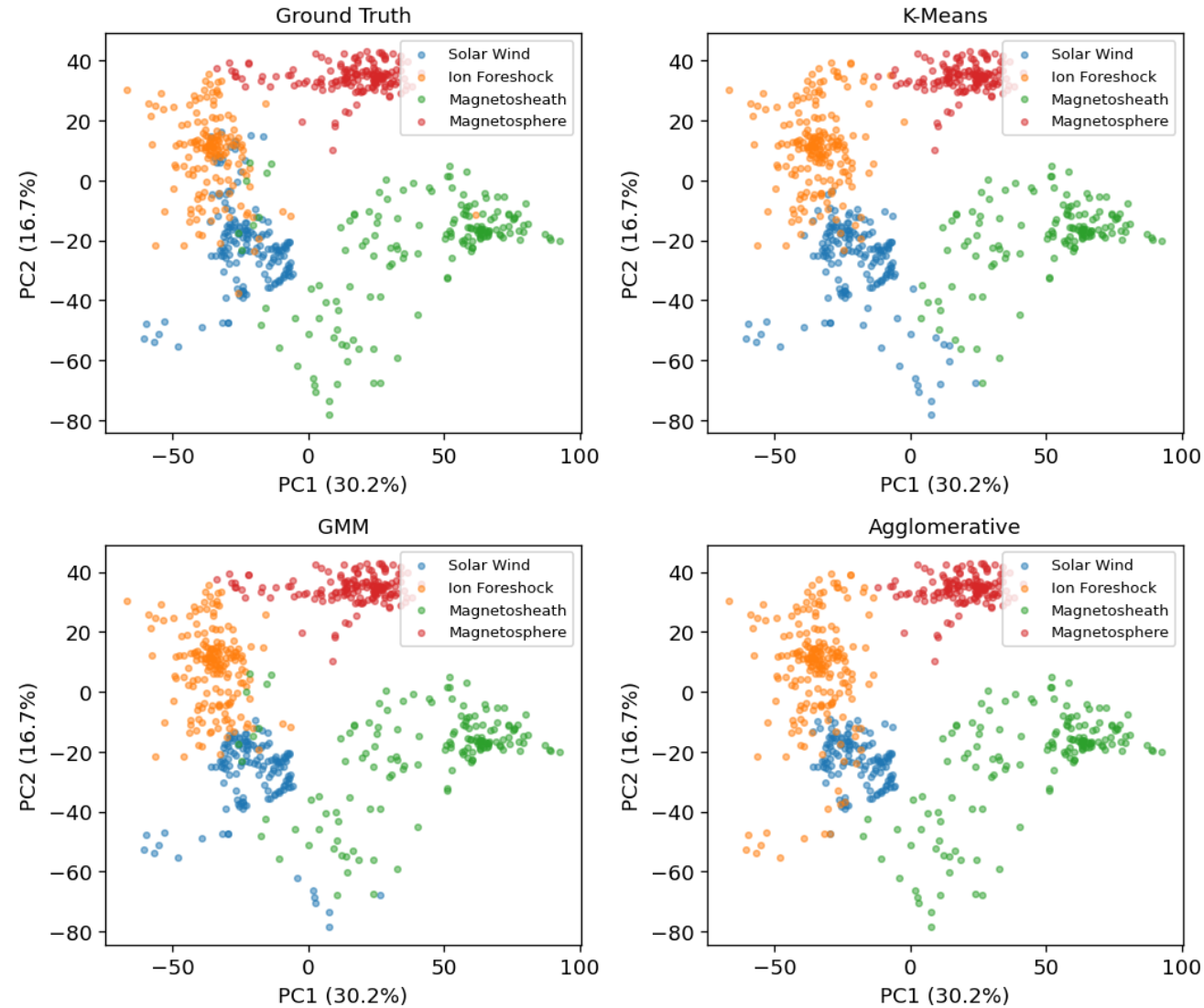


Why this beats a single accuracy number

- It reveals which mistakes are made and how often - actionable, not just a grade

Clustering Results

PC1-PC2: ground truth vs cluster assignments (test set)

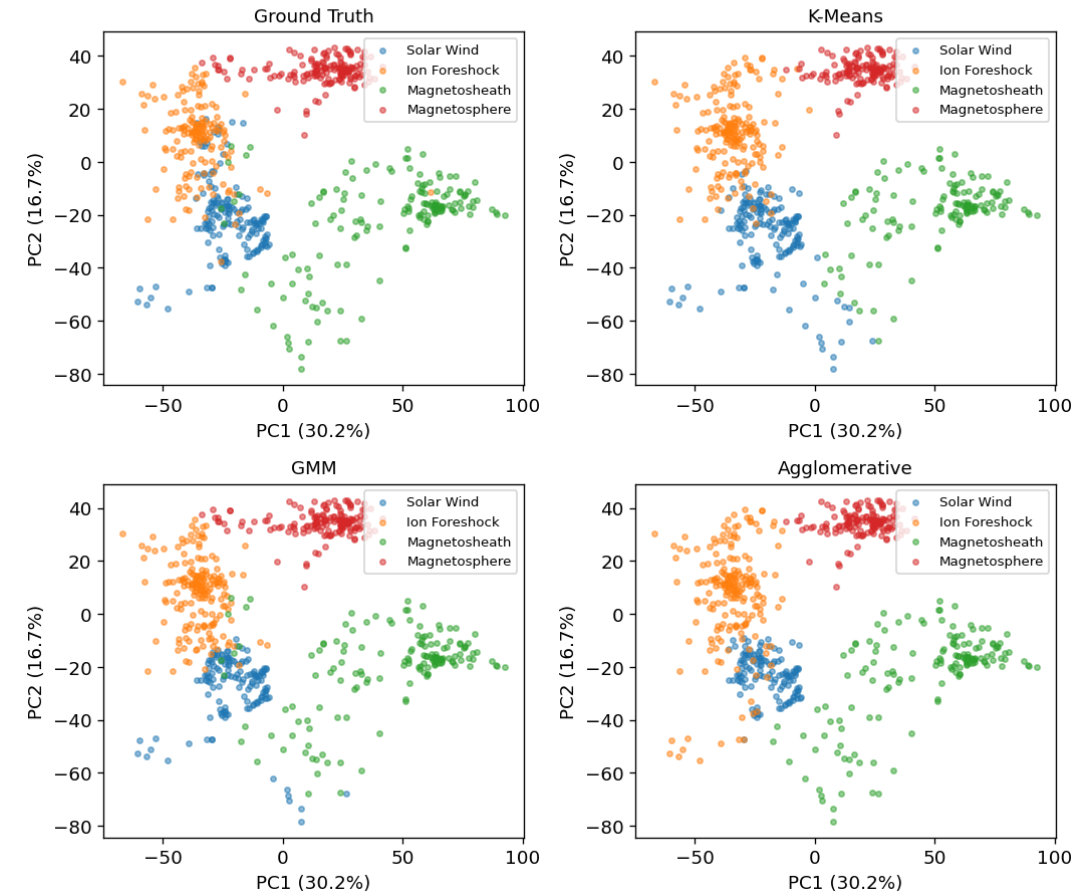


Summary

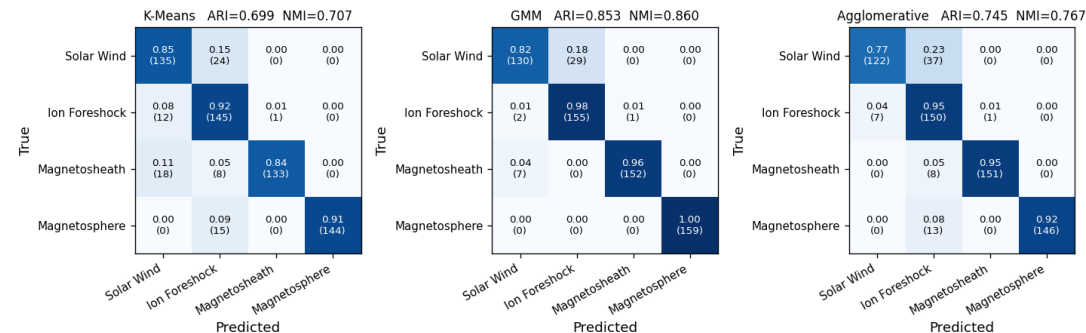
1. Split data into training, (validation) and test.
2. Reduce features with PCA
3. Transform to PCA coordinate frame
4. Cluster of reduced features

Basic clustering mechanism perform reasonably well at distinguishing between inside or outside the magnetosphere.

PC1-PC2: ground truth vs cluster assignments (test set)



Confusion matrices (row-normalised) — test set, Hungarian label matching



Section 3: Supervised Classification - CNN

Notebook: 04_cnn_plasma_classification

Supervised vs Unsupervised:

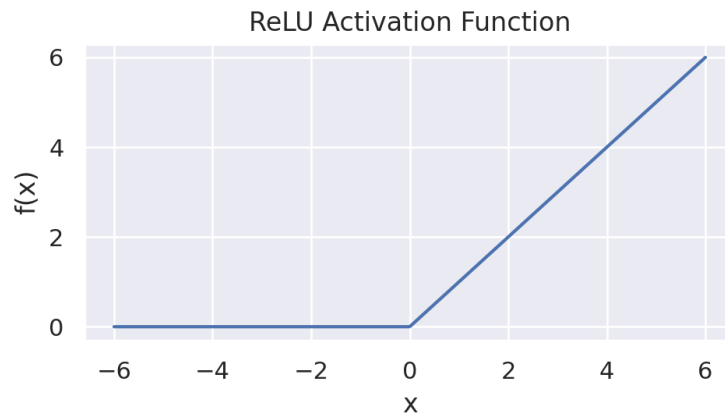
What changes when we use labels?

- Clustering: find structure. CNN: learn the mapping from spectrogram to label, using the labels
- New power: the loss function directly optimises what you care about - classification accuracy
- New responsibility: labels must be trustworthy. Garbage labels = garbage model

CNN Architecture Walk-Through

Convolutional Neural Networks (CNN)

- Convolutional layers
 - Learn feature maps
 - Analogous to PCA transform
- Fully connected layers
 - Learn classification based on learned features
- Non-linear activation functions
 - Enables learning non-linear relationships
 - Here we use ReLU



- Softmax on network output:
 - Gives classification probabilities

```
import torch.nn as nn

class CNN(nn.Module):
    """Small CNN for 2-D ion spectrogram classification.

    Input: (batch, 1, N, n_bins) - single-channel spectrogram
    Output: (batch, n_classes) - raw logits
    """
    def __init__(self, n_time: int, n_bins: int,
                  n_classes: int = 4, dropout: float = 0.3):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 4, kernel_size=3, padding=0),
            nn.ReLU(),
            nn.Conv2d(4, 4, kernel_size=3, padding=0),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2),

            dummy = torch.zeros(1, 1, n_time, n_bins)
            flat_size = self.features(dummy).numel()

            self.classifier = nn.Sequential(
                nn.Flatten(),
                nn.Linear(flat_size, 128),
                nn.ReLU(),
                nn.Linear(128, n_classes),
                nn.Softmax(dim=-1),

        def forward(self, x: torch.Tensor) -> torch.Tensor:
            return self.classifier(self.features(x))

model = CNN(n_time=66,
            n_bins=32,
            n_classes=4)
```


Preparing the dataset

Splitting into Training/Validation/Test sets

- Ration 60/10/30 % (keep test set size)
- Adding Validation:
 - For tuning and early stopping

No stratify support in Torch, we'll use Scikit-learn, `train_test_split(...)`

1. First split full dataset into training, and tmp
2. Split tmp into Validation and Test
3. Use index to devide torch Dataset
4. Create Loaders

```
VAL_SIZE = 0.10
TEST_SIZE = 0.30
_, y = ds_2d[:]
idx = np.arange(len(y))

idx_train, idx_tmp = train_test_split(
    idx, test_size=TEST_SIZE + VAL_SIZE, stratify=y,
    random_state=RANDOM_STATE,
)

y_tmp = y[idx_tmp]
relative_val_size = VAL_SIZE / (TEST_SIZE + VAL_SIZE)
idx_test, idx_val = train_test_split(
    idx_tmp, test_size=relative_val_size, stratify=y_tmp,
    random_state=RANDOM_STATE,
)

ds_train = Subset(ds_2d, idx_train)
ds_val = Subset(ds_2d, idx_val)
ds_test = Subset(ds_2d, idx_test)

BATCH_SIZE = 32
train_loader = DataLoader(ds_train, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(ds_val, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(ds_test, batch_size=BATCH_SIZE, shuffle=False)
```

Training the Network

Loss function

- The function we optimize for
- **CrossEntropyLoss**
 - Common in classification tasks
 - Minimizes the distance between probability distributions

Optimizer

- Updates the network parameters based on the gradients.
- Common optimizers: **SDG** or **ADAM**

```
N_EPOCHS = 50
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimiser = optim.Adam(model.parameters(), lr=1e-4)

train_loss, test_loss = [], []
for epoch in range(1, N_EPOCHS + 1):
    model.train()
    run_loss, total = 0.0, 0
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
        optimiser.zero_grad()
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        loss.backward()
        optimiser.step()
        run_loss += loss.item() * len(y_batch)
        total += len(y_batch)

    train_loss.append(run_loss / total)

    model.eval()
    run_loss, total = 0.0, 0
    with torch.no_grad():
        for X_batch, y_batch in test_loader:
            X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            run_loss += loss.item() * len(y_batch)
            total += len(y_batch)
    test_loss.append(run_loss / total)
```

Training the Network

The Training Loop

- Training
 - Load a batch of training data
 - Reset gradients
 - Run data through the model
 - Calculate loss
 - Calculate gradients
 - Update network weights
- Testing
 - Load a batch of test data
 - Run data through the model
 - Calculate loss
 - No gradients, No backwards step.

```
N_EPOCHS = 50
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimiser = optim.Adam(model.parameters(), lr=1e-4)

train_loss, test_loss = [], []
for epoch in range(1, N_EPOCHS + 1):
    model.train()
    run_loss, total = 0.0, 0
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
        optimiser.zero_grad()
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        loss.backward()
        optimiser.step()
        run_loss += loss.item() * len(y_batch)
        total += len(y_batch)

    train_loss.append(run_loss / total)

    model.eval()
    run_loss, total = 0.0, 0
    with torch.no_grad():
        for X_batch, y_batch in test_loader:
            X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            run_loss += loss.item() * len(y_batch)
            total += len(y_batch)
    test_loss.append(run_loss / total)
```

Overfitting and Underfitting

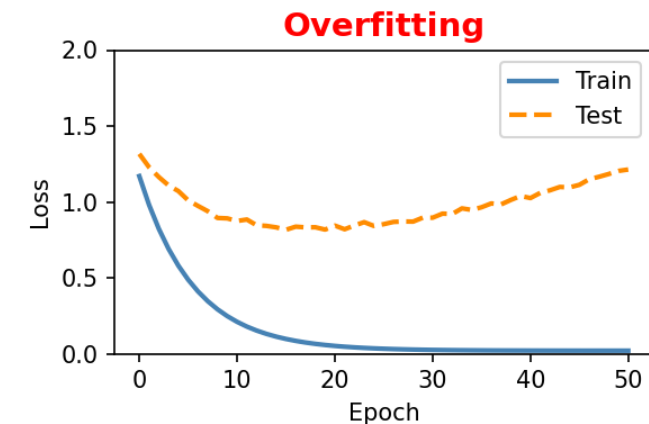
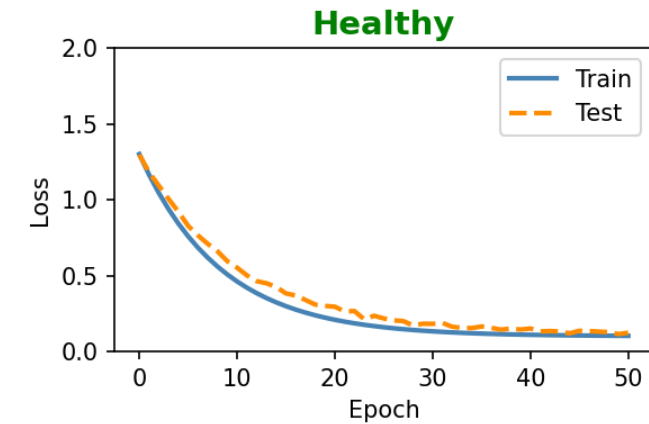
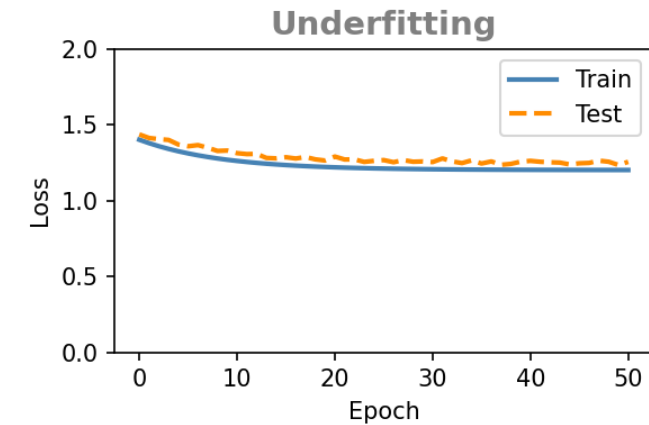
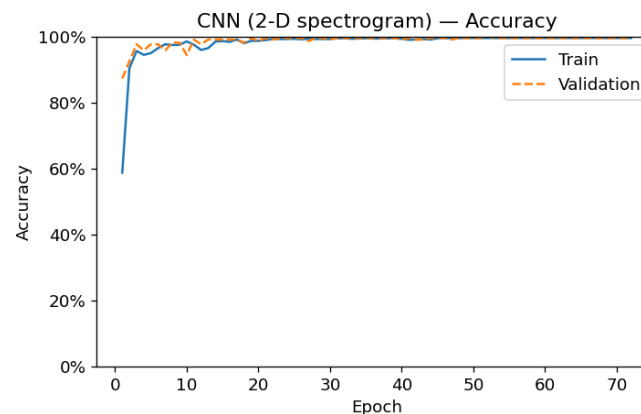
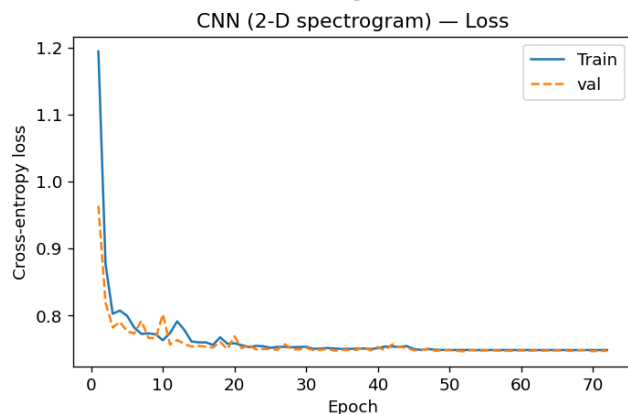
The two failure modes of any ML model

- Overfitting: memorizes training data, fails on new data.
- Underfitting: model too simple to capture structure.

Diagnosing from training curves

- Train approx. Test loss -> healthy generalization
- Train much lower than Test loss -> overfitting.
 - Fix: more data, dropout, weight decay, early stopping
- Train approx. Test loss, both high -> underfitting.
 - Fix: bigger model, more epochs, better features

Our network training:



Classification Metrics: Beyond Accuracy

Accuracy alone is misleading

From the confusion matrix, for each class:

Precision = $TP / (TP + FP)$: Of windows predicted as region X, how many really were X?

Recall = $TP / (TP + FN)$: Of windows that were region X, how many did we catch?

F1 Score = $2 * P * R / (P + R)$: Harmonic mean of precision and recall

Macro F1

- Averages F1 across all classes equally - treats rare Ion Foreshock the same as common Magnetosheath
- Good choice here: we care equally about the correct identification of all four regions

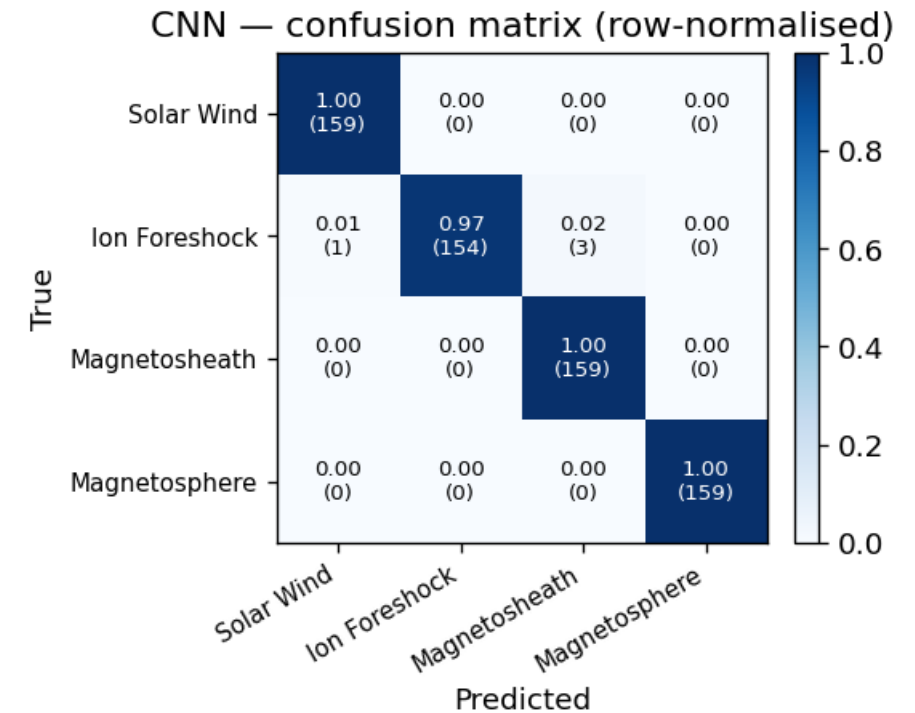
When each metric matters more

- Precision: when false alarms are costly (e.g. flagging non-events as reconnection)
- Recall: when missing events is unacceptable (e.g. detecting every magnetopause crossing)

CNN: Metrics and Confusion

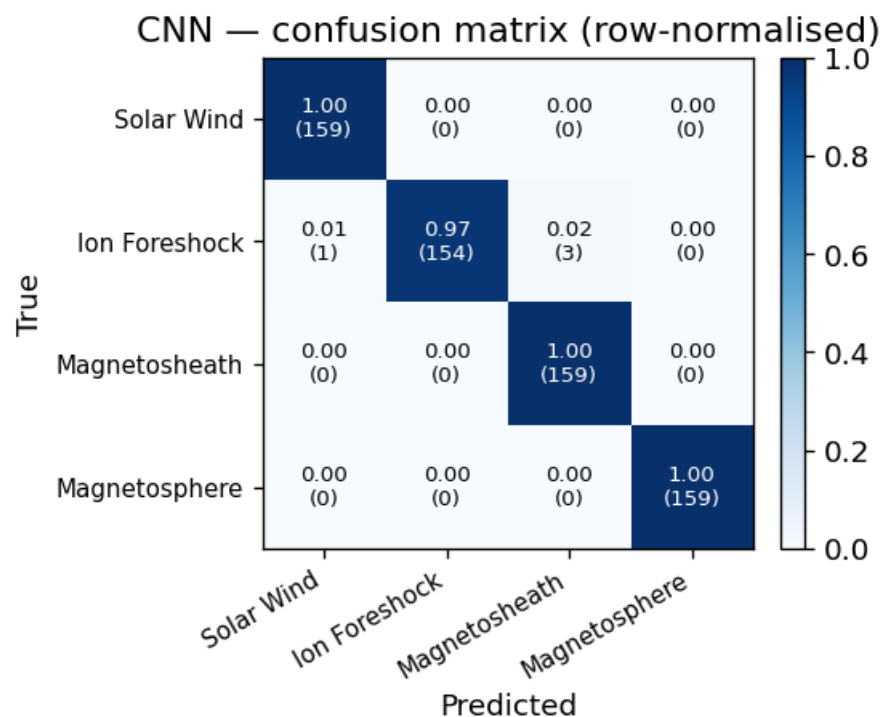
Accuracy: 0.99

	Precision	Recall	F1-score
Solar Wind	0.99	1.00	1.00
Ion Foreshock	0.99	0.97	0.98
Magnetosheath	0.99	1.00	1.00
Magnetosphere	1.00	1.00	1.00

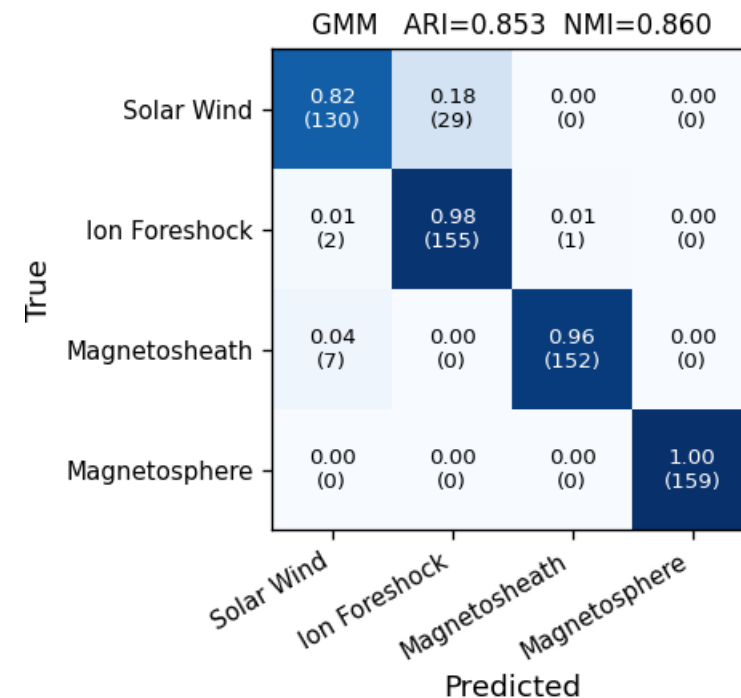


Caution: Training, validation, and test sets are small and very clean!

Clustering vs CNN



Trainable parameters: 223,040



Trainable parameters: 19,008 + 134

Key Takeaways

Know your data; The data shapes the method, feature reductions can reveal structure

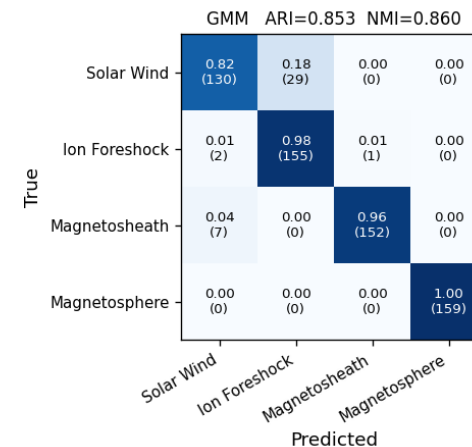
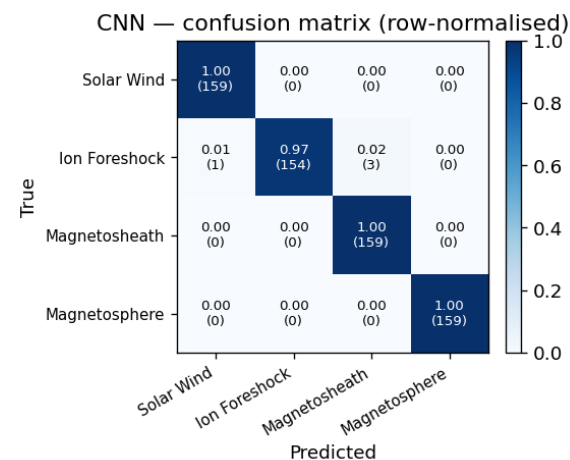
Preprocessing matters:

- Scale to reveal features, per-feature standardization can remove feature correlations.
- Fit preprocessing on training data only,
- Balance your data
- Preprocessing shapes everything downstream

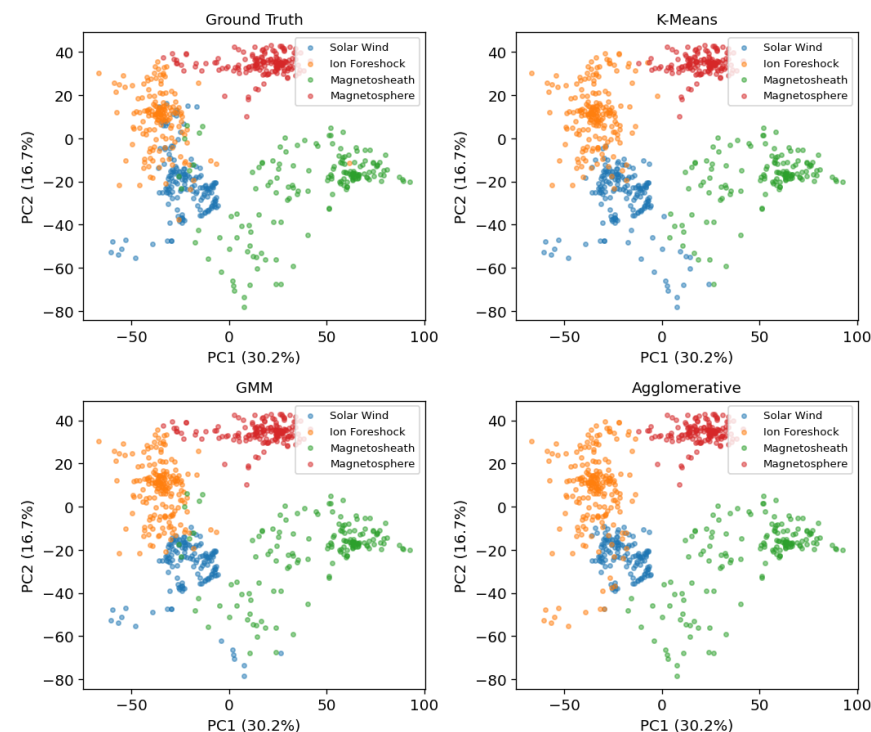
Unsupervised clustering works surprisingly well: PCA + GMM achieves $ARI \approx 0.87$ with no labels.

Supervised learning optimizes for what you care about, but encodes choices: garbage labels produce a garbage model.

Read your confusion matrix: A near-perfect diagonal is the goal, but the off-diagonal entries are where the science is.



PC1-PC2: ground truth vs cluster assignments (test set)



Afternoon Tutorials:



We are here:
Gustaf Dahlander,
Teknikringen 31, Floor 3

This afternoon we will be here:
Germund Dahlquist,
Lindstedtsvägen 3, Floor 5

Afternoon Tutorials:

Three tracks:

- A: Create an unlabeled dataset of the Ion Spectrum, evaluate clusters. Train CNN and MLP on a labeled dataset and classify your dataset.
- B: Create a labeled dataset with new features (raw measurements, engineered features, and/or statistical features), evaluate the dataset, look at clustering and NN classification.
- C: Create a dataset for a mission of your choice, using a feature of your choice. Evaluate clustering methods. Consider using Autoencoders as an unsupervised NN method.

Notebooks are available on the GitHub repo.